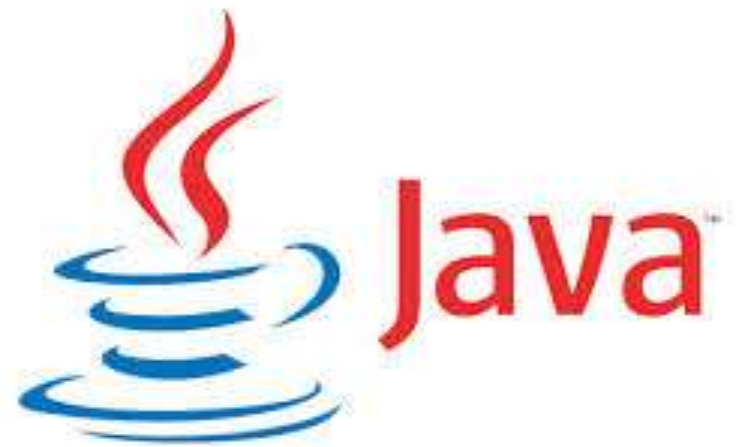




SOC2030

Application Programming in Java



Week 14 Java Collections API

Dr. Andrei Dragunov

Details

- Arrays
- The Java Collections Framework
- Collections Interfaces
- Concrete Collections
- Iterating through Collections
- Lists
- Queues

Arrays

An array is a group of variables (called **elements** or **components**) containing values that all have the *same* type.

Arrays are *objects*, so they're considered *reference types*.

As you'll soon see, what we typically think of as an array is actually a *reference* to an array object in memory.

The *elements* of an array can be either *primitive types* or *reference types*.

To refer to a particular element in an array, we specify the *name* of the reference to the array and the *position number* of the element in the array. The position number of the element is called the element's **index** or **subscript**.

Logical Array Representation

A program refers to any one of these elements with an **array-access expression** that includes the *name* of the array followed by the *index* of the particular element in **square brackets** (`[]`).

Name of array (c) →	c[0]	-45
	c[1]	6
	c[2]	0
	c[3]	72
	c[4]	1543
	c[5]	-89
	c[6]	0
	c[7]	62
	c[8]	-3
	c[9]	1
	c[10]	6453
Index (or subscript) of the element in array c →	c[11]	78

Declaring and Creating Arrays

Array objects occupy space in memory. Like other objects, arrays are created with keyword `new`. To create an array object, you specify the type of the array elements and the number of elements as part of an **array-creation expression** that uses keyword `new`. Such an expression returns a *reference* that can be stored in an array variable.

```
int[] c; // declare the array variable  
c = new int[12]; // create the array; assign to array variable
```

Using an Array Initializer

You can create an array and initialize its elements with an **array initializer**—a comma-separated list of expressions (called an **initializer list**) enclosed in braces. In this case, the array length is determined by the number of elements in the initializer list.

```
int[] n = { 10, 20, 30, 40, 50 };
```

Passing Arrays to Methods

- To pass an array argument to a method, specify the name of the array *without any brackets*. For example, if array `hourlyTemperatures` is declared as

```
double[] hourlyTemperatures = new double[24];
```

- then the method call

```
modifyArray(hourlyTemperatures);
```

- For a method to receive an array reference through a method call, the method's parameter list must specify an *array parameter*.

```
void modifyArray(double[] b)
```

Pass-By-Value vs. Pass-By-Reference

Two ways to pass arguments in method calls in many programming languages are **pass-by-value** and **pass-by-reference** (sometimes called **call-by-value** and **call-by-reference**).

When an argument is passed by value, a *copy* of the argument's *value* is passed to the called method. The called method works exclusively with the copy.

Changes to the called method's copy do *not* affect the original variable's value in the caller.

Variable-Length Argument Lists

- With **variable-length argument lists**, you can create methods that receive an unspecified number of arguments. A type followed by an **ellipsis (...)** in a method's parameter list indicates that the method receives a variable number of arguments of that particular type.
- This use of the ellipsis can occur only *once* in a parameter list, and the ellipsis, together with its type and the parameter name, must be placed at the *end* of the parameter list.

Class Arrays

Class **Arrays** helps you avoid reinventing the wheel by providing static methods for common array manipulations. These methods include **sort** for *sorting* an array, **binarySearch** for *searching* a *sorted* array, **equals** for *comparing* arrays and **fill** for *placing values into an array*. These methods are overloaded for primitive-type arrays and for arrays of objects.

Multidimensional Arrays

- Multidimensional arrays with two dimensions are often used to represent *tables* of values with data arranged in *rows* and *columns*. To identify a particular table element, you specify *two* indices.
- *By convention*, the first identifies the element's row and the second its column. Arrays that require two indices to identify each element are called **two-dimensional arrays**.

Two-dimensional array with three rows and four columns.

	Column 0	Column 1	Column 2	Column 3
Row 0	<code>a[0][0]</code>	<code>a[0][1]</code>	<code>a[0][2]</code>	<code>a[0][3]</code>
Row 1	<code>a[1][0]</code>	<code>a[1][1]</code>	<code>a[1][2]</code>	<code>a[1][3]</code>
Row 2	<code>a[2][0]</code>	<code>a[2][1]</code>	<code>a[2][2]</code>	<code>a[2][3]</code>

Column index
Row index
Array name

Arrays of One-Dimensional Arrays

Like one-dimensional arrays, multidimensional arrays can be initialized with array initializers in declarations. A two-dimensional array `b` with two rows and two columns could be declared and initialized with **nested array initializers** as follows:

```
int[][] b = {{1, 2}, {3, 4}};
```

The initial values are *grouped by row* in braces. So 1 and 2 initialize `b[0][0]` and `b[0][1]`, respectively, and 3 and 4 initialize `b[1][0]` and `b[1][1]`, respectively.

Two-Dimensional Arrays with Rows of Different Lengths

The manner in which multidimensional arrays are represented makes them quite flexible. In fact, the lengths of the rows in array `b` are *not* required to be the same. For example,

```
int[][] b = {{1, 2}, {3, 4, 5}};
```

creates integer array `b` with two elements (determined by the number of nested array initializers) that represent the rows of the two-dimensional array. Each element of `b` is a *reference* to a one-dimensional array of `int` variables.

Creating Two-Dimensional Arrays with Array-Creation Expressions

A multidimensional array with the *same* number of columns in every row can be created with an array-creation expression.

```
int[][] b = new int[3][4];
```

Programs can also use variables to specify array dimensions, because *new* creates arrays at execution time—not at compile time. The elements of a multidimensional array are initialized when the array object is created. A multidimensional array in which each row has a *different* number of columns can be created as follows:

```
int[][] b = new int[2][];    // create 2 rows  
b[0] = new int[5]; // create 5 columns for row 0  
b[1] = new int[3]; // create 3 columns for row 1
```

Lists, Stacks, Queues, and Priority Queues

The Java API provides several predefined data structures, called **collections**, used to store groups of related objects in memory. These classes provide efficient methods that organize, store and retrieve your data *without* requiring knowledge of how the data is being *stored*.

This reduces application-development time.

Collection

*The **Collection** interface defines the common operations for lists, vectors, stacks, queues, priority queues, and sets.*

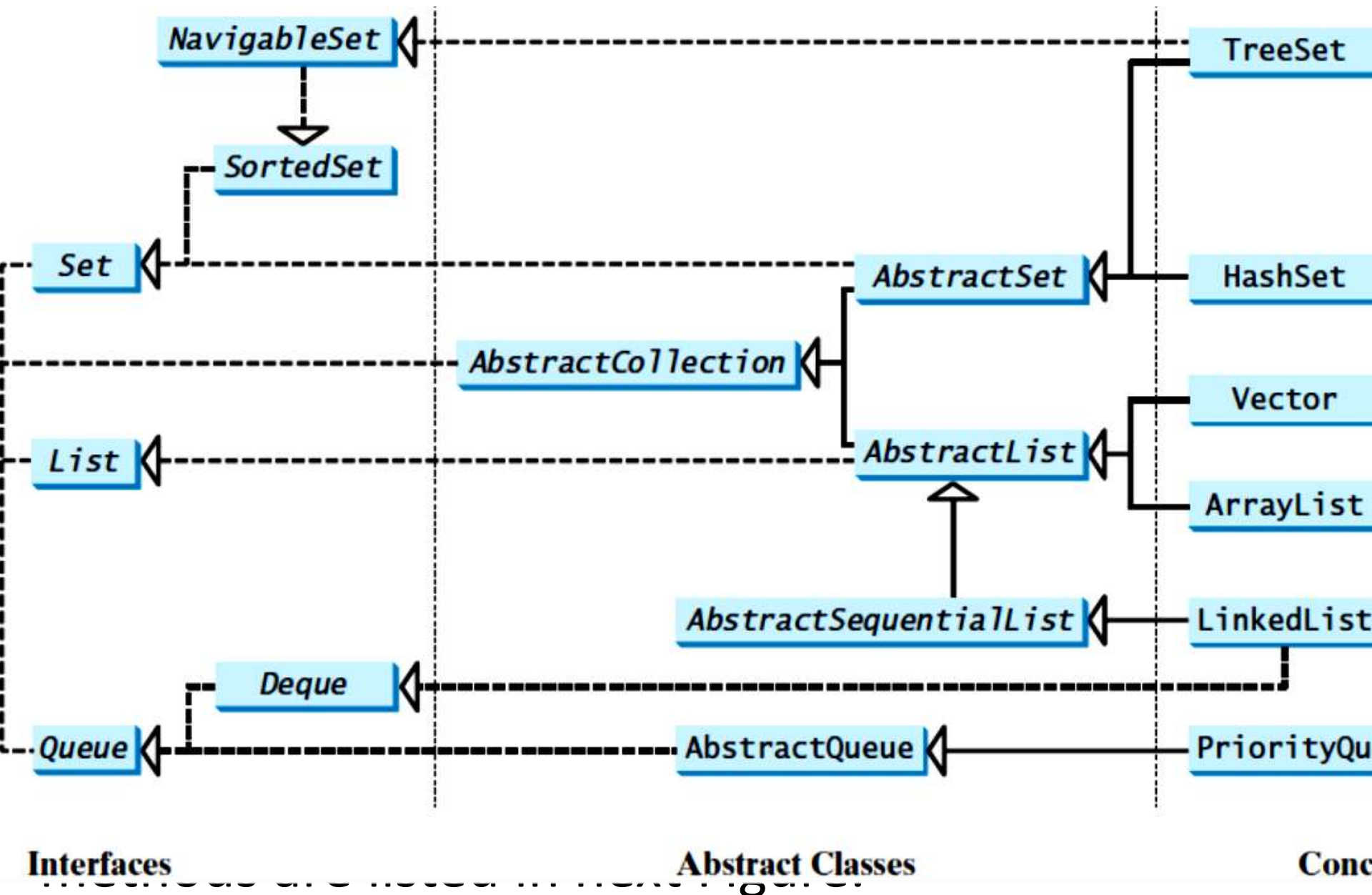
The Java Collections Framework supports two types of containers:

- One for storing a collection of elements is simply called a ***collection***.
- The other, for storing key/value pairs, is called a ***map***.

Collections

- **Sets** store a group of nonduplicate elements.
- **Lists** store an ordered collection of elements.
- **Stacks** store objects that are processed in a last-in, first-out fashion.
- **Queues** store objects that are processed in a first-in, first-out fashion.
- **PriorityQueues** store objects that are processed in the order of their priorities.

Class Hierarchy



«interface»
java.util.Collection<E>

+add(o: E): boolean
+addAll(c: Collection<? extends E>)
+clear(): void
+contains(o: Object): boolean
+containsAll(c: Collection<?>): boolean
+equals(o: Object): boolean
+hashCode(): int
+isEmpty(): boolean
+remove(o: Object): boolean
+removeAll(c: Collection<?>): boolean
+retainAll(c: Collection<?>): boolean
+size(): int
+toArray(): Object[]

Adds a new element o to this collection.

Adds all the elements in the collection c to this collection.

Removes all the elements from this collection.

Returns true if this collection contains the element o.

Returns true if this collection contains all the elements in c.

Returns true if this collection is equal to another collection o.

Returns the hash code for this collection.

Returns true if this collection contains no elements.

Removes the element o from this collection.

Removes all the elements in c from this collection.

Retains the elements that are both in c and in this collection.

Returns the number of elements in this collection.

Returns an array of Object for the elements in this collection.

ArrayList<T> (package java.util)

The collection class **ArrayList<T>** (package java.util) can *dynamically* change its size to accommodate more elements. The T (by convention) is a *placeholder*—when declaring a new ArrayList, replace it with the type of elements that you want the ArrayList to hold. For example,

```
ArrayList<String> list;
```

declares list as an ArrayList collection that can store only Strings.

Classes with this kind of placeholder that can be used with any type are called **generic classes**. *Only nonprimitive types can be used to declare variables and create objects of generic classes.* However, Java provides a mechanism—known as *boxing*—that allows primitive values to be wrapped as objects for use with generic classes. So, for example,

```
ArrayList<Integer> integers;
```

declares `integers` as an `ArrayList` that can store only `Integers`. When you place an `int` value into an `ArrayList<Integer>`, the `int` value is *boxed* (wrapped) as an `Integer` object, and when you get an `Integer` object from an `ArrayList<Integer>`, then assign the object to an `int` variable, the `int` value inside the object is *unboxed* (unwrapped).

Some methods and properties of class ArrayList<T>.

Method	Description
add	Adds an element to the <i>end</i> of the ArrayList.
clear	Removes all the elements from the ArrayList.
contains	Returns true if the ArrayList contains the specified element; otherwise, returns false.
get	Returns the element at the specified index.
indexOf	Returns the index of the first occurrence of the specified element in the ArrayList.
remove	Overloaded. Removes the first occurrence of the specified value or the element at the specified index.
size	Returns the number of elements stored in the ArrayList.
trimToSize	Trims the capacity of the ArrayList to the current number of elements.

Iterators

*Each collection is **Iterable**. You can obtain its **Iterator** object to traverse all the elements in the collection.*

Iterator is a classic design pattern for walking through a data structure without having to expose the details of how data is stored in the data structure. The **Collection** interface extends the **Iterable** interface. The **Iterable** interface defines the **iterator** method, which returns an iterator. The **Iterator** interface provides a uniform way for traversing elements in various types of collections.

- The **iterator()** method in the **Iterable** interface returns an instance of **Iterator**, which provides sequential access to the elements in the collection using the **next()** method. You can also use the **hasNext()** method to check whether there are more elements in the iterator, and the **remove()** method to remove the last element returned by the iterator.

Lists

- *The **List** interface extends the **Collection** interface and defines a collection for storing elements in a sequential order. To create a list, use one of its two concrete classes: **ArrayList** or **LinkedList**.*
- We used **ArrayList** to test the methods in the **Collection** interface in the preceding sections. Now we will examine **ArrayList** in more depth. We will also introduce another useful list – **LinkedList**.

Diamond (<>) Notation for Creating an Object of a Generic Class

```
LinkedList<Object> linkedList = new LinkedList<>(arrayList);
```

Java SE 7 introduced the **diamond (<>) notation** to simplify statements like this. Using <> in a class instance creation expression for an object of a *generic* class tells the compiler to determine what belongs in the angle brackets. In Java SE 7 and higher, the preceding statement can be written as:

```
LinkedList<Object> linkedList = new LinkedList<>();
```

When the compiler encounters the diamond (<>) in the class instance creation expression, it uses the declaration of variable items to determine the List's element type (String)—this is known as *inferring the element type*.

Vector and Stack Classes

- **Vector** is a subclass of **AbstractList**, and **Stack** is a subclass of **Vector** in the Java API.

java.util.Vector<E>



java.util.Stack<E>

+Stack()
+empty(): boolean
+peek(): E
+pop(): E
+push(o: E): E
+search(o: Object): int

Creates an empty stack.

Returns true if this stack is empty.

Returns the top element in this stack.

Returns and removes the top element in this stack.

Adds a new element to the top of this stack.

Returns the position of the specified element in this stack.

Queues and Priority Queues

A *queue* is a first-in, first-out data structure. Elements are appended to the end of the queue and are removed from the beginning of the queue. In a *priority queue*, elements are assigned priorities. When accessing elements, the element with the highest priority is removed first.

«interface»
java.util.Queue<E>

```
+offer(element: E): boolean  
+poll(): E  
  
+remove(): E  
  
+peek(): E  
  
+element(): E
```

Inserts an element into the queue.

Retrieves and removes the head of this queue, or `null` if this queue is empty.

Retrieves and removes the head of this queue and throws an exception if this queue is empty.

Retrieves, but does not remove, the head of this queue, returning `null` if this queue is empty.

Retrieves, but does not remove, the head of this queue, throws an exception if this queue is empty.